

Practical week-11

Task 1 : Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
vennela_07@VennelaPC:~$ mkdir pthread_counter
cd pthread_counter
vennela_07@VennelaPC:~/pthread_counter$ nano counter.c
vennela_07@VennelaPC:~/pthread_counter$ gcc counter.c -o counter
-pthread
vennela_07@VennelaPC:~/pthread_counter$ ./counter
Initial counter value: 0
Expected counter value: 500000
Actual counter value: 298425
vennela_07@VennelaPC:~/pthread_counter$ ./counter
Initial counter value: 0
Expected counter value: 500000
Actual counter value: 211198
vennela_07@VennelaPC:~/pthread_counter$ ./counter
Initial counter value: 0
Expected counter value: 500000
Actual counter value: 500000
vennela_07@VennelaPC:~/pthread_counter$ ./counter
Initial counter value: 0
Expected counter value: 500000
Actual counter value: 269752
vennela_07@VennelaPC:~/pthread_counter$ ./counter
Initial counter value: 0
Expected counter value: 500000
Actual counter value: 500000
```

```
GNU nano 7.2 counter.c
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

int counter = 0;

void *increment_counter(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
```

```

pthread_t threads[NUM_THREADS];

printf("Initial counter value: %d\n", counter);

for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_create(&threads[i], NULL, increment_counter, NULL);
}

for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

printf("Expected counter value: %d\n",
        NUM_THREADS * INCREMENTS);

printf("Actual counter value: %d\n", counter);

return 0;
}

```

Task 2 : Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```

vennela_07@VennelaPC:~$ mkdir pthread_mutex
cd pthread_mutex
vennela_07@VennelaPC:~/pthread_mutex$ nano counter_mutex.c
vennela_07@VennelaPC:~/pthread_mutex$ gcc counter_mutex.c -o counter_mutex -pthread
vennela_07@VennelaPC:~/pthread_mutex$ ./counter_mutex
Initial counter value: 0
Expected counter value: 500000
Actual counter value: 500000
vennela_07@VennelaPC:~/pthread_mutex$ ./counter_mutex
Initial counter value: 0
Expected counter value: 500000
Actual counter value: 500000

```

```

GNU nano 7.2 counter_mutex.c
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

int counter = 0;
pthread_mutex_t lock;

void *increment_counter(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    pthread_mutex_init(&lock, NULL);

    printf("Initial counter value: %d\n", counter);

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    pthread_create(&threads[1], NULL, increment_counter, NULL);
}

for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

printf("Expected counter value: %d\n",
      NUM_THREADS * INCREMENTS);

printf("Actual counter value: %d\n", counter);

pthread_mutex_destroy(&lock);

return 0;
}

```